🧪Hands-On Lab Exercise 7: Functions in Shell Script

Topic: Understanding and Using Functions in Shell Script
Author: Dr. Sandeep Kumar Sharma

A function in shell scripting is a block of code that is written once and can be used many times in a script. Instead of writing the same commands again and again, we place them inside a function and call that function whenever we need that work to be done. This makes the script clean, structured, easy to understand, and easy to manage. Functions help in organizing large scripts into small logical parts, just like chapters in a book. Each function performs a specific task, and together they form a complete automation solution.

In real system automation, scripts become very long and complex. Without functions, such scripts become difficult to read, difficult to debug, and difficult to maintain. Functions solve this problem by breaking the script into meaningful sections. For example, one function can handle installation, another can handle configuration, and another can handle system checks. This separation of work makes automation professional, structured, and scalable.

We need functions because automation is not just about writing commands, it is about building systems. Functions make scripts reusable, because the same logic can be reused multiple times without rewriting code. They improve readability, because anyone reading the script can understand what each function does. They improve maintainability, because changes can be made in one place and the entire script gets updated. Functions also make scripts reliable, because logic is centralized instead of being scattered.

In simple words, a function is like a small machine inside a big machine. The big machine (script) calls the small machines (functions) to perform different tasks. Each small machine has a specific job, and together they complete the full work.

📊Learning Objective

- Understand the concept of functions
- Learn why functions are needed
- Learn how to define a function
- Learn how to write function logic
- Learn how to call a function
- Learn how to use multiple functions in a script

🔗Learning Outcome

After this lab, participants will: - Create functions in shell scripts - Use functions for automation - Build structured scripts - Write reusable code - Organize scripts professionally

---

📘How to Write a Function in Shell Script

A function is written like this:

```
function_name() {
   commands
}
```

This means: - `function_name` is the name of the function
- `{ }` contains the logic of the function
- Commands inside `{ }` are the function body

📌How to Call a Function

To call a function, simply write its name:

```
function_name
```

When the function name is written, the system executes all commands inside that function.

🧪Example 1: Simple Function Script

Step 1: Create Script

```
touch func1.sh
```

Step 2: Open File

```
nano func1.sh
```

Step 3: Write Script

```
#!/bin/bash

welcome() {
  echo "Welcome to Shell Scripting"
  echo "This is your first function"
}

# calling the function
welcome
```

Step 4: Permission

```
chmod +x func1.sh
```

Step 5: Run

```
./func1.sh
```

🧪 Example 2: Multiple Functions in One Script

Step 1: Create Script

```
touch func2.sh
```

Step 2: Open File

```
nano func2.sh
```

Step 3: Write Script

```
#!/bin/bash

create_dirs() {
  mkdir project
  mkdir project/app
  mkdir project/logs
  mkdir project/config
  echo "Project directories created"
}

create_files() {
  touch project/app/app.txt
  touch project/logs/log.txt
  touch project/config/config.txt
  echo "Project files created"
}

show_message() {
  echo "Setup completed successfully"
}

# calling functions
create_dirs
create_files
show_message
```

Step 4: Permission

```
chmod +x func2.sh
```

Step 5: Run

```
./func2.sh
```

🧪 Example 3: Function-Based System Info Script

Step 1: Create Script

```
touch func3.sh
```

Step 2: Open File

```
nano func3.sh
```

Step 3: Write Script

```
#!/bin/bash

show_date() {
  echo "Date and Time:"
  date
}

show_user() {
  echo "Current User:"
  whoami
}

show_path() {
  echo "Working Directory:"
  pwd
}

show_disk() {
  echo "Disk Usage:"
  df -h
```

```
}

# calling functions
show_date
show_user
show_path
show_disk
```

Step 4: Permission

```
chmod +x func3.sh
```

Step 5: Run

```
./func3.sh
```

Understanding the Flow

The script does not execute randomly. It first reads all function definitions. Then, when a function name is called, the system executes the commands written inside that function. This allows clear flow control and structured execution. Each function behaves like a small program inside the main program.

Key Understanding

Functions make scripts structured. Functions make scripts reusable. Functions make scripts readable. Functions make scripts professional. Functions are the foundation of large automation systems.

📌Summary

Functions convert scripts into systems. Instead of writing long unstructured scripts, functions create organized automation. This is how real DevOps tools, automation frameworks, and infrastructure scripts are written.

💴End of Lab

This lab introduces the **architecture of automation**.

Now scripts are not just scripts — they are **structured systems**.

Functions are the foundation of: - Large automation tools - DevOps scripts - Cloud automation - Infrastructure automation - CI/CD systems

---

Author:
Dr. Sandeep Kumar Sharma

🔗**LinkedIn:**
linkedin.com/in/sandeep-kaushik-2a345856

Medium Blog:
medium.com/@shyamsandeep28